

```
Livox MID-360 Documentation Bundle :root { --ink: #1f2933; --muted: #52606d; --line:
#d9e2ec; --panel: #f8fbff; --accent: #0b6e4f; --accent-soft: #e6f4ef; } body { margin: 0;
font-family: "Segoe UI", Roboto, Helvetica, Arial, sans-serif; color: var(--ink);
background: linear-gradient(180deg, #eef6ff 0%, #ffffff 18%); } main { max-width: 980px;
margin: 0 auto; padding: 40px 24px 64px; } h1, h2, h3 { line-height: 1.2; color: #123; }
h1 { font-size: 2.3rem; margin-bottom: 0.3rem; } h2 { margin-top: 2.2rem; padding-bottom:
0.35rem; border-bottom: 2px solid var(--line); } h3 { margin-top: 1.4rem; } p, li {
line-height: 1.58; } .lead { color: var(--muted); font-size: 1.05rem; max-width: 54rem; }
.note, .card { background: var(--panel); border: 1px solid var(--line); border-radius:
12px; padding: 14px 16px; } .note { border-left: 6px solid var(--accent); background:
var(--accent-soft); } .grid { display: grid; grid-template-columns: repeat(auto-fit,
minmax(240px, 1fr)); gap: 12px; } table { width: 100%; border-collapse: collapse; margin:
1rem 0; font-size: 0.95rem; } th, td { border: 1px solid var(--line); padding: 0.6rem
0.7rem; vertical-align: top; } th { text-align: left; background: #eef4fa; } pre {
background: #10161d; color: #eef4fa; padding: 14px 16px; border-radius: 10px; overflow-x:
auto; } code { font-family: "SFMono-Regular", Consolas, "Liberation Mono", Menlo,
monospace; } ul { padding-left: 1.2rem; }
Livox MID-360 Documentation Bundle
```

Consolidated local documentation for the Livox MID-360 stack in this repository. It combines the repos HTML guides, quick-start notes, Python SDK wrappers, viewer and SLAM entry points, and the sample network configuration.

Primary local sources used for this bundle: docs/lidar_docs.html, docs/lidar_cheatsheet.html, docs/geoff_stack.md, scripts/navigation/slam/livox2_python.py, scripts/navigation/slam/livox_python.py, scripts/sensors/live_points.py, scripts/navigation/slam/live_slam.py, scripts/navigation/slam/mid360_config.json.

1. What This Stack Contains

SDK 2 Wrapper

livox2_python.py exposes a minimal push-mode wrapper around Livox-SDK2 and converts packet data into NumPy XYZ arrays.

SDK 1 Wrapper

livox_python.py auto-discovers Livox devices on the subnet, connects, starts sampling, and forwards frames to Python callbacks.

Live Viewer

live_points.py is the first-line smoke test for streaming point clouds into Open3D.

SLAM Demo

live_slam.py feeds Livox scans into KISS-ICP and renders a growing map and pose estimate.

2. Architecture Overview

LayerPurposeLocation

Livox-SDK2 / Livox-SDK Device discovery, UDP negotiation, packet decoding, Cartesian point conversion. External vendor libraries installed under system library paths.

Python ctypes wrappers Load the shared libraries and expose point packets as NumPy arrays through `handle_points()`. `scripts/navigation/slam/livox2_python.py`, `scripts/navigation/slam/livox_python.py`

Visualization Open3D windows for live preview and map rendering. `scripts/sensors/live_points.py`, `scripts/navigation/slam/live_slam.py`

Odometry / mapping KISS-ICP registration, pose estimation, and voxel map maintenance. `scripts/navigation/slam/live_slam.py`

The broader Geoff architecture also routes Livox MID-360 data over Ethernet/UDP into the laptop-side visualization and control stack. The repo documentation describes this path as:

Livox MID-360 -> `livox2_python.py` -> network -> `lidar_reader` / GUI consumers

3. Network and Host Requirements

- Connect the G-1 shoulder Ethernet port to the host PC.
 - Assign the host to the 192.168.123.0/24 subnet, typically 192.168.123.222.
 - Verify reachability of the LiDAR at 192.168.123.120.
 - Allow or disable firewall rules that would block Livox UDP traffic.
- ```
sudo ip addr add 192.168.123.222/24 dev <your_nic>
ping 192.168.123.120
```

#### Traffic Ports Mentioned In Repo Docs

MID-360 config ports 56100, 56200, 56300, 56400, 56500 plus matching host-side receive ports

Troubleshooting note from repo docs Open UDP 55000-56000 and 7500-7510 if packet flow stalls in the SDK path

### 4. One-Time Installation

Recommended: Livox-SDK2

```
git clone https://github.com/Livox-SDK/Livox-SDK2.git
cd Livox-SDK2
mkdir build
cd build
cmake .. -DCMAKE_BUILD_TYPE=Release
make -j$(nproc)
sudo make install
sudo ldconfig
```

Legacy Fallback: Livox-SDK

```
git clone https://github.com/Livox-SDK/Livox-SDK.git
```

```
cd Livox-SDK
mkdir build
cd build
cmake .. -DCMAKE_BUILD_TYPE=Release
make -j$(nproc)
sudo make install
sudo ldconfig
```

## Python Environment

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r REQUIREMENTS.txt
```

The local docs and scripts expect at least numpy, open3d, and kiss-icp for the SLAM path.

## 5. SDK Wrapper Behavior

SDK 2: livox2\_python.py

- Prefers liblivox\_lidar\_sdk\_shared.so or liblivox\_lidar\_sdk.so.
  - Accepts overrides through LIVOX\_SDK2\_LIB or LIVOX\_SDK2\_DIR.
  - Initializes the SDK with a JSON config path and host IP.
  - Registers a point-cloud callback before starting SDK threads.
  - On info-change, forces normal work mode, enables point sending, and selects Cartesian high point data.
  - Aggregates many UDP packets into pseudo-frames before calling handle\_points().
- Aggregation defaults in the wrapper constructor:

```
frame_time=0.20
frame_packets=120
```

Supported packet types currently handled in this wrapper:

- data\_type == 1: Cartesian High, converted from millimeters to meters.
- data\_type == 2: Cartesian Low, converted from centimeters to meters.

SDK 1: livox\_python.py

- Searches for liblivox\_sdk.so or LivoxSdk.dll.
- Supports overrides through LIVOX\_SDK\_LIB and LIVOX\_SDK\_DIR.
- Uses broadcast discovery and device-state callbacks.
- Connects to every detected LiDAR on the subnet and starts sampling.
- Forwards Cartesian packet formats to handle\_points() as float32 XYZ meters.

## 6. MID-360 JSON Configuration

The repo ships a minimal sample at scripts/navigation/slam/mid360\_config.json. It points the host side to 192.168.123.222 and uses the standard MID-360 port layout.

```
{
 "MID360": {
 "lidar_net_info": {
 "cmd_data_port": 56100,
```

```

"push_msg_port": 56200,
"point_data_port": 56300,
"imu_data_port": 56400,
"log_data_port": 56500
},
"host_net_info": [
{
"host_ip": "192.168.123.222",
"multicast_ip": "224.1.1.5",
"cmd_data_port": 56101,
"push_msg_port": 56201,
"point_data_port": 56301,
"imu_data_port": 56401,
"log_data_port": 56501
}
]
}
}
}

```

The quick-start guide also references a fuller ROS-driver-flavored configuration that includes:

- lidar\_summary\_info.lidar\_type = 8
  - lidar\_configs[0].ip = 192.168.123.120
  - pcl\_data\_type = 1
  - Optional extrinsic roll, pitch, yaw, x, y, z fields
7. Running the Local Python Demos

#### Live Point Cloud Preview

```

cd scripts/sensors
python3 live_points.py

```

This script prefers SDK2 and falls back to SDK1 if needed. It keeps a ring buffer of recent frames in Open3D to reduce flicker and applies mount correction if the sensor is upside down.

#### LiDAR SLAM

```

cd scripts/navigation/slam
python3 live_slam.py

```

For outdoor tuning:

```
LIVOX_PRESET=outdoor python3 live_slam.py
```

Internal flow documented in the repo:

```

[SDK callback thread] [main / UI thread]
Livox.handle_points -> KissICP.register_frame -> viewer.push()
^ |
+----- viewer.tick() <-----+

```

## 8. Runtime Controls and Environment Variables

### VariableEffect

LIVOX\_MOUNT=normal|upside\_downControls the 180 degree X-axis correction. Repo defaults to upside\_down.

LIVOX\_PRESET=indoor|outdoorSelects KISS-ICP and frame aggregation tuning in the SLAM script.

LIDAR\_TILT\_DEGApplies a fixed inverse tilt correction to compensate for a physically tilted sensor.

LIDAR\_TILT\_AXIS=x|y|zSelects the tilt compensation axis. Default is y.

LIVOX\_SDK2\_LIB, LIVOX\_SDK2\_DIRExplicit SDK2 shared library override paths.

LIVOX\_SDK\_LIB, LIVOX\_SDK\_DIRExplicit SDK1 shared library override paths.

## 9. SLAM Presets

### ParameterIndoorOutdoor

frame\_time0.35 s0.20 s

frame\_packets200120

voxel\_size0.4 m1.0 m

max\_range30 m120 m

downsample\_limit5,000,000 points3,000,000 points

min\_motion0.03 m0.10 m

conv\_criterion5e-51e-4

max\_iters800500

## 10. ROS Driver Quick Start

The repo also carries a ROS Noetic quick-start for livox\_ros\_driver2:

```
source /opt/ros/noetic/setup.bash
mkdir -p ~/catkin_ws/src
cd ~/catkin_ws/src
git clone https://github.com/Livox-SDK/livox_ros_driver2.git
cd livox_ros_driver2
./build.sh ROS1
cd ~/catkin_ws
catkin_make -DCMAKE_BUILD_TYPE=Release -DROS_EDITION=ROS1
```

Example launch:

```
roslaunch livox_ros_driver2 msg_MID360.launch \
user_config_path:=/home/$USER/livox_cfg/MID360_config.json \
xfer_format:=0 data_src:=0
```

Verification commands from the local quick-start:

```
rostopic hz /livox/lidar
sudo tcpdump -i <your_nic> udp port 56300 -n -c 5
```

## 11. Troubleshooting Checklist

- Only one network interface should sit on 192.168.123.0/24.
- If bind or init fails, check the JSON host IP and port ownership.
- If frames stall, verify raw UDP arrival with tcpdump.
- If SDK libraries are not found, set the appropriate LIVOX\_SDK\* environment variable or install the vendor SDK system-wide.
- If KISS-ICP import fails, upgrade it with pip install --upgrade kiss-icp.
- If the Open3D viewer looks empty or white, zoom out or reset the view.
- If point clouds appear inverted, set LIVOX\_MOUNT=normal or correct tilt values.
- If ROS tools cannot see the topic, confirm the ROS setup files were sourced and the driver was built for ROS1.

## 12. Relevant Local Files

- docs/lidar\_docs.html
- docs/lidar\_cheatsheet.html
- docs/geoff\_stack.md
- scripts/navigation/slam/livox2\_python.py
- scripts/navigation/slam/livox\_python.py
- scripts/navigation/slam/live\_slam.py
- scripts/sensors/live\_points.py
- scripts/navigation/slam/mid360\_config.json